

5. Izrazi niza

Izraz niza možete koristiti da biste dobili ili postavili vrednost elementa niza.

Izraz niza se sastoji od identifikatora (ime niza) nakon čega sledi indeks, koji je par zagrada [] koje sadrže indeks niza, poput imena[1].

Imajte na umu da:

- Kada koristite indeks da biste postavili vrednost na novom indeksu, nova stavka će biti dodata nizu na ovom indeksu.

- Ako niz ne postoji, biće kreiran prvi put kada postavite stavku u njemu. Ali ako pokušate da pročitate stavku iz niza pre nego što je kreirate, dobićete grešku.

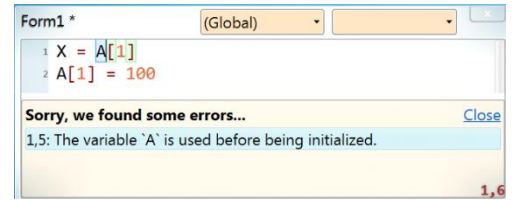
- Ako koristite indeks da biste postavili vrednost na već postojeći indeks, ovo će promeniti vrednost stavke koja postoji na ovom indeksu.

- Možete koristiti indeks niza za rad sa nizom znakova.

Za više informacija, pogledajte metode `Text.GetCharacterAt` i `Text.SetCharacterAt`.

- Takođe možete koristiti indeks niza da biste pristupili cifri broja, gde možete pročitati vrednost ove cifre ili je podesiti:

```
Num = 478
TW.WriteLine("Poslednja cifra je: " & Num[3])
Num[2] = 5
TW.WriteLine(Num)
Num[1] = 12
TW.WriteLine(Num)
```



```
Last digit is: 8
458
1258
```

- Indeks niza može biti numerički literal kao što je 1, 2, .. itd. Na primer:

```
A[1] = 10 ' Napravite niz i dodajte prvu stavku
A[2] = 15 ' Dodajte drugu stavku
A[2] = 20 ' Izmenite drugu stavku
A[3] = 30 ' Dodajte treću stavku
A[4] = 40 ' Dodajte četvrtu stavku
A[5] = 50 ' Dodajte petu stavku
```

- Indeks takođe može biti ključ string literala kao što je „id“. Ključ niza ne razlikuje velika i mala slova, tako da su „ID“, „Id“ i „id“ zapravo isti ključ. Na primer:

```
Student["ime"] = "Adam"
Student["godine"] = 18
Student["pol"] = "muški"
TextWindow.WriteLine({
Student["Ime"],      ' Adam
Student["GODINE"],   ' 18
Student["pol"]       ' muški
})
```

- Ako je string ključ validan identifikator (kao u gornjem primeru), sVB vam dozvoljava da koristite operator pretraživanja rečnika ! da biste mu direktno pristupili:

```
TextWindow.WriteLine({
Student!Ime,
Student!Godine,
Student!pol
})
```

Sintaksa ključa je detaljna, ali fleksibilnija, jer možete koristiti bilo koji string (kao što je "moje ime" ili ";") kao ključ.

- Takođe možete koristiti operator pretraživanja rečnika da biste dodali stavke:

```
Ime!Studenta = "Adam"
```

```
Godine!Studenta = 18
Pol!Studenta = "muški"
```

U ovom slučaju, niz Student izgleda kao dinamički objekat (Exando objekat u VB.NET-u), sa tri dinamička svojstva: Ime, Godine i Pol.

- Moguće je koristiti 0 ili čak negativan broj kao indeks!

Podrazumevano, indeks sVB niza počinje od 1 i to je ono što treba da očekujete kada dobijete niz iz bilo koje metode sVB bibliotečkih tipova, ali pošto možete koristiti ključeve nizova kao indeksere, validno je koristiti "0" i "-1" kao ključeve, a pošto sVB vrši implicitnu konverziju između brojeva i nizova, možete direktno koristiti 0 i -1! na primer:

```
X["-1"] = "Test"
TextWindow.WriteLine(X[-1]) ' Test
```

- Možete koristiti promenljivu ili bilo koji izraz da biste dobili indeks ili ključ elementa. Ovo je ono što obično radimo u for petljama:

```
For I = 1 To 5
TextWindow.WriteLine(A[I])
Next
```

```
10
20
30
40
50
```

- Možete koristiti operator = da biste proverili jednakost dva niza, gde se smatraju jednakima ako imaju isti broj elemenata, pri čemu svaki ključ ima istu vrednost u svakom od njih. Na primer:

```
X["jedan"] = 1
X["dva"] = 2
Y["Dva"] = 2
Y["Jedan"] = 1
A = {1, 2}
B = {2, 1}
C = {1, 2}
TextWindow.WriteLine({
X = Y, ' Tačno
X = A, ' Netačno
Y = A, ' Netačno
A = B, ' Netačno
A = C ' Tačno
})
```

- sVB zaključuje da je izraz niza tipa Array. Ovo vam omogućava da koristite promenljivu a kao objekat niza i da direktno iz nje pristupate metodama tipa Array, kao što je:

```
TextWindow.WriteLine(A.Count) ' 0
```

Što je skraćeni oblik ovog ekvivalentnog koda:

```
TextWindow.WriteLine(Array.GetItemCount(A))
```

- Možete implicitno podeliti red posle leve zagrade [i pre desne zagrade]. Na primer:

```
X[
Listbox1.SelectedIndex
] = Listbox1.SelectedItem
```

Niz ili rečnik?

Činjenica da se nizu pristupa pomoću ključeva znači da je on zapravo rečnik i može sadržati praznine, pa ako pokušate da pročitate vrednost stavke koja ne postoji, izraz niza će vratiti prazan string. Ali neke postojeće stavke mogu zapravo sadržati prazan string, pa kako razlikovati ova dva slučaja? Rešenje je korišćenje metode Array.ContainsIndex da biste saznali da li stavka postoji ili ne pre nego što pokušate da je pročitate.

Niz takođe može imati neuređene indekse, kao što su:

```
A[5] = 1  
A[1] = 2  
A[2] = 3
```

Dakle, kada pozovete metodu `Array.GetAllIndices` ili svojstvo `Array.Indices` da biste dobili niz koji sadrži sve ključeve niza, dobićete indekse redosledom kojim ste ih definisali. Sledeći kôd će za gornji niz ispisati {5, 1, 2}:

```
TextWindow.WriteLine(Array.ToStr(A.Indices))
```

Dakle, možete koristiti taj indeksni niz da biste napisali petlju `for` za čitanje svih elemenata niza!

To ponašanje je nasleđeno iz Small Basic-a, ali sVB vam to malo olakšava dodavanjem metode `Array.GetKeyAt`, ali najlakši način je da koristite petlju `ForEach` da biste prošli kroz sve elemente niza bez obzira na njihove ključeve. Na primer:

```
ForEach Item In A  
TextWindow.WriteLine(Item)  
Next
```

Višedimenzionalni niz

Možete koristiti više od jednog indeksa nakon imena niza, svaki od njih se bavi odgovarajućim podnizom. Na primer, sledeći niz sadrži podatke 3 studenta, gde je svaki podatak o studentu podniz koji sadrži 3 stavke (njegovo ime, godine i pol):

```
Student[1]["ime"] = "Adam"  
Student[1]["godine"] = 18  
Student[1]["pol"] = "muški"  
Student[2]["ime"] = "Jovan"  
Student[2]["godine"] = 17 - 128 -  
Student[2]["pol"] = "muški"  
Student[3]["ime"] = "Nora"  
Student[3]["godine"] = 17  
Student[3]["pol"] = "ženski"
```

Možete koristiti dve ugneždene petlje da biste prikazali sve podatke o studentima ovako:

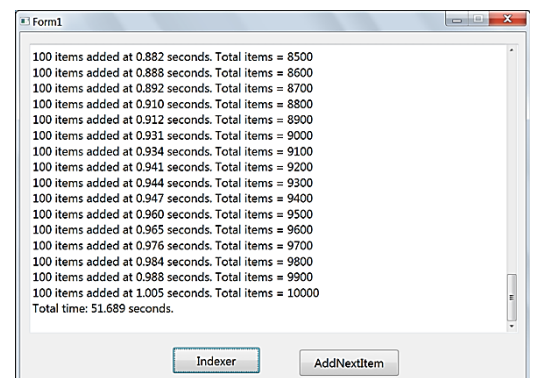
```
For I = 1 To 3  
ForEach Item In Student[I]  
TextWindow.WriteLine(Item)  
Next  
TextWindow.WriteLine("")  
Next
```

```
Adam  
18  
male  
  
John  
17  
male  
  
Nora  
17  
female
```

Performanse nizova

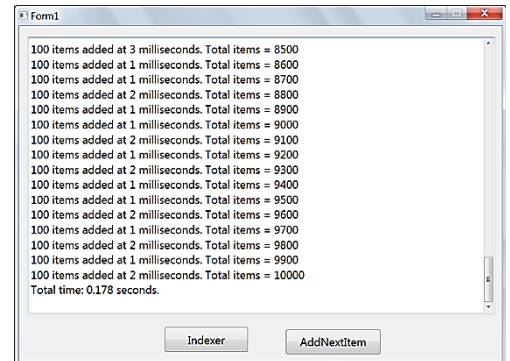
Da bi bio jednostavan za decu i početnike kao obrazovni programski jezik, Small Basic kompajler i njegov sistem tipova su dizajnirani zbog jednostavnosti, a ne zbog najboljih performansi.

Tip niza je očigledan primer za to. Svaki put kada koristite indeks niza da biste dodali stavku u niz ili promenili vrednost postojeće stavke, kreira se novi niz, a stare vrednosti niza se kopiraju u njega, a zatim se promena primenjuje na novi niz!



Ovo su naravno najgore performanse ikada u pogledu potrošnje memorije i brzine aplikacije, ali ne bi trebalo da bude toliko loše kada radite sa malim nizom, gde u većini slučajeva ne očekujete da pređe 20 stavki. Ali uticaj na performanse će se eksponencijalno povećavati sa svakim novim elementom koji dodate, zbog kopiranja svih prethodno dodatih stavki prvo! Da biste ovo videli svojim očima, otvorite projekat „Performanse niza“ u folderu sa primerima, pritisnite F5 da biste pokrenuli projekat i kliknite na dugme Indeksir. Ovo dugme dodaje 10.000 stavki u niz, što će potrajati. Tekstualno polje će se ažurirati svakih 100 stavki, kako bi vam se pokazalo vreme potrebno za njihovo dodavanje, gde ćete primetiti da svakih 100 stavki traje duže, gde će poslednjih 100 stavki biti oko 54 puta duže od prvih 100 stavki! Ukupno vreme će biti prikazano nakon što se proces završi. Na mom starom računaru, ovo je trajalo oko 52 sekunde!

Dobra vest je da je sVB obezbedio rešenje za ovo, korišćenjem metoda `Array.Append` i `Array.SetItemAt`. Ove dve metode dodaju i menjaju elemente niza po referenci, bez potrebe za kreiranjem novog niza i kopiranjem starog u njega. Ovo je super brzo, a možete to isprobati klikom na dugme `SetItemAt` u „Performansama niza“, što dodaje 10.000 elemenata nizu za manje od 0,2 sekunde na mom starom računaru (uporedite to sa 52 sekunde da ih indeksir doda!).



Možda me pitate ovde: Zašto sVB ne koristi metode `Array.Append` i `Array.SetItemAt` za direktno izvršavanje indeksera? Nažalost, nije tako jednostavno! Operacije referenciranjem mogu biti opasne, jer mogu izmeniti drugi niz dok ne obraćate pažnju! Ovo se može desiti ako ste dobili niz na jedan od ovih načina:

1. Kopiranje niza pomoću operatora `=`.

2. Slanje niza parametru funkcije

3. Primanje niza iz povratne vrednosti funkcije. Niz koji ste dobili na ove načine je samo referenca na originalni niz, i zato sVB kreira novi niz pre dodavanja ili modifikovanja bilo koje stavke. Ali kada koristite metode `Array.Append` i `Array.SetItemAt` za promenu niza, promena će se desiti na vašem nizu i originalnom nizu jer imaju istu referencu (isti prostor za skladištenje u memoriji)! Na primer:

```
X = {1, 2, 3}
Y = X
X.Append(4)
Y[5] = 5
Y.Append(6)
TextWindow.WriteLine({X.ToStr(), Y.ToStr()})
```

```
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
```

Očigledno je da se dodavanje četvrte stavke korišćenjem metode `Y.Append` pojavilo u oba niza, jer je izvršavanje naredbe `Y = X` učinilo da `X` i `Y` imaju istu referencu (pokazuju na isti niz u memoriji), dok je korišćenje indeksera za dodavanje pete stavke kreiralo novi niz i dodalo novu stavku u njega. Nakon toga, korišćenje metode `Y.Append` neće imati efekta na `X`, jer `Y` ukazuje na drugi niz.

Mislim da sada shvatate koliko ovo može biti zbunjujuće!

Moj savet je da koristite metode `Array.Append` i `Array.SetItemAt` na novom nezavisnom nizu koji sami kreirate, i kao jednostavnu zaštitu, uvek se pobrinite da koristite ovu liniju:

```
a = {}
```

pre nego što počnete da koristite `a.Append` i `a.SetItemAt`.

Kao što vidite, operacije sa referencom nisu nešto što treba upoznati ni sa manjim ni sa početnicima, i zato je sVB zadržao ponašanje indeksa nizova nasleđeno od SB, jer je pogodno za male uzorke i lagane aplikacije gde performanse nisu problem.

Napomena: ako metode `Array.StItemAt` ili `Array.Append` ne rade, verovatno je to zato što ste im prosledili tekstualnu reprezentaciju niza, a ne stvarni niz. Ovo se veoma retko dešava, jer sVB automatski konvertuje tekst u niz. Ali ako ste se nekako susreli sa tako retkim problemom, možete ga lako popraviti korišćenjem metode `Text.ToArray` da biste dobili niz iz njegove tekstualne reprezentacije.

Izrazi inicijalizatora niza

Možete koristiti par zagrada `{}` da biste postavili više elemenata niza odjednom:

```
x = {1, 2, 3}
```

Ugnežđeni inicijalizatori su takođe podržani kada radite sa višedimenzionalnim nizovima (nazubljenim nizovima):

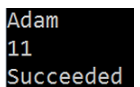
```
y = {"a", "b", {1, 2, 3}}
```

Takođe možete koristiti promenljive ili povratne vrednosti funkcija unutar inicijalizatora. Na primer, gornji kôd se može prepisati kao:

```
y = {"a", "b", x}
```

I možete poslati inicijalizator kao argument funkciji:

```
TextWindow.WriteLine({"Adam", 11, "Succeeded"})
```



Adam
11
Succeeded

Imajte na umu da dodeljivanje inicijalizatora niza promenljivoj čini da sVB zaključuje da je tipa `Array`. Zbog toga vam ponekad može biti potreban prazan inicijalizator da biste kreirali prazan niz:

```
a = {}
```

Ovo vam omogućava da koristite promenljivu `a` kao objekat niza i da direktno iz nje pristupite metodama tipa `Array` kao što je:

```
TextWindow.WriteLine(A.Count) ' 0
```

Što je skraćeni oblik ovog ekvivalentnog koda:

```
TextWindow.WriteLine(Array.GetItemCount(A))
```

Takođe imajte na umu da možete koristiti implicitni kontinuitet linije da biste formatirali inicijalizator niza na prilično čitljiv način:

```
y = {  
    "a",  
    "b",  
    {1, 2, 3}  
}
```

ili:

```
y = {  
    "a", "b",  
    {1, 2, 3}  
}
```

ili:

```
y = {  
    "a",  
    "b", {
```

```
1,  
2,  
3  
}}
```

ili:

```
Y = {  
  "a",  
  "b", {  
    1,  
    2,  
    3  
  }  
}
```

Imate slobodu da možete da podelite red na ovim mestima:

- posle bilo koje početne zagrade {,
- posle bilo kog zareza,
- i pre bilo koje zatvarajuće zagrade },

Samo podelite red na validnom mestu, a sVB uređivač koda će se pobrinuti za uvlačenje reda umesto vas.